

= QUALITY ASSURANCE

Multi-Model Consistency: Ensure Every Model Agrees

When you route queries across GPT-4o, Claude, and Gemini, the same user can get contradictory answers on different sessions. Consistency checking ensures all routed models produce trustworthy, aligned responses.

4 consistency layers covered in this carousel



SEMANTIC SCORING



NLI FACTUAL CHECK



SCHEMA VALIDATION



REGRESSION SUITE

Production Python code. Real NLI models. Real jsonschema enforcement.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost

Why Multi-Model Consistency Matters

A routed multi-model system sends different users to different LLMs. When those models disagree, users get contradictory information. This silently destroys trust, especially in regulated domains like healthcare, finance, and legal.

≠ Failure Mode 1: Factual Contradiction

User A (routed to GPT-4o) asks: "Does our product support OAuth 2.0?" Answer: Yes.
User B (routed to Claude) asks the same: Answer: No. Both answers are confidently stated. One is wrong. Support tickets spike the next morning.

🚫 Failure Mode 2: Refusal Inconsistency

GPT-4o refuses a borderline request. Claude answers it. Same query, same user base, different behavior. This is a safety policy violation, not just a quality issue. It can create legal and compliance exposure.

📦 Failure Mode 3: Schema Mismatch

Your frontend expects a JSON object with fields "summary" and "confidence". GPT-4o returns it. Gemini sometimes returns plain text or different field names. The frontend crashes for 30% of users randomly.



ConsistencyScore Dataclass

```
from dataclasses import dataclass
from typing import Literal

@dataclass
class ConsistencyScore:
    query: str
    answers: dict[str, str] # model_name → answer text
    semantic_score: float # 0.0 - 1.0, higher = more consistent
    factual_score: float # 0.0 = contradiction found
    schema_valid: bool
    verdict: Literal["CONSISTENT", "INCONSISTENT", "DEGRADED"]

    def overall_score(self) → float:
        # Weighted: semantic 40%, factual 40%, schema 20%
        return (
            self.semantic_score * 0.40 +
            self.factual_score * 0.40 +
            (1.0 if self.schema_valid else 0.0) * 0.20
        )

    def is_consistent(self) → bool:
        return self.verdict == "CONSISTENT"
```

✓ CONSISTENT

All checks pass. Overall score above 0.85. Safe to route users to any model in the pool.

⚠ INCONSISTENT

Factual contradiction or semantic score below 0.75. Route to primary model only until resolved.



score_semantic_consistency Pairwise Cosine

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

SEMANTIC_THRESHOLD = 0.85

def score_semantic_consistency(
    answers: dict[str, str],
    embedder,
) → tuple[float, str]:
    models = list(answers.keys())
    texts = list(answers.values())
    embs = np.array([embedder.embed(t) for t in texts])
    sim_mat = cosine_similarity(embs)
    n = len(embs)

    # Mean of upper triangle (all unique pairs)
    pairs = [(sim_mat[i][j], models[i], models[j])
              for i in range(n)
              for j in range(i + 1, n)]
    avg_sim = np.mean([p[0] for p in pairs])

    verdict = "CONSISTENT" if avg_sim ≥ SEMANTIC_THRESHOLD else "INCONSISTENT"
    low = [f"{p[1]} vs {p[2]}: {p[0]:.3f}" for p in pairs if p[0] < SEMANTIC_THRESHOLD]
    if low: log(f"Low-similarity pairs: {low}")
    return round(float(avg_sim), 4), verdict
```

 Semantic similarity catches paraphrase drift and topic divergence between models. It does not catch factual contradictions where two models state opposite facts using similar vocabulary. Use NLI checking for that.



check_factual_consistency NLI Contradiction Detection

```
from sentence_transformers import CrossEncoder

nli = CrossEncoder("cross-encoder/nli-deberta-v3-small")
# Labels: 0=contradiction, 1=neutral, 2=entailment

def check_factual_consistency(
    answers: dict[str, str]
) → tuple[float, list[str]]:
    models = list(answers.keys())
    texts = list(answers.values())
    contradictions = []

    for i in range(len(texts)):
        for j in range(i + 1, len(texts)):
            a, b = texts[i], texts[j]
            # Check both directions
            score_ab = nli.predict([(a, b)])[0]
            score_ba = nli.predict([(b, a)])[0]

            is_contradiction = (
                score_ab.argmax() == 0 or
```

🔊 Why DeBERTa-v3?

deberta-v3-small runs in under 50ms per pair on CPU. It understands logical contradiction, not just keyword mismatch. "The product supports X" vs "The product does not support X" is correctly flagged even if cosine similarity between the two sentences is above 0.90.



validate_output_contract jsonschema Enforcement

```
import json, jsonschema

OUTPUT_SCHEMA = {
    "type": "object",
    "required": ["summary", "confidence", "sources"],
    "properties": {
        "summary": {"type": "string", "minLength": 10},
        "confidence": {"type": "number", "minimum": 0, "maximum": 1},
        "sources": {"type": "array", "items": {"type": "string"}},
    },
    "additionalProperties": False
}

def validate_output_contract(
    answers: dict[str, str]
) → dict[str, list[str]]:
    errors = {}
    for model, raw in answers.items():
        try:
            parsed = json.loads(raw)
            jsonschema.validate(parsed, OUTPUT_SCHEMA)
            errors[model] = []
        except (json.JSONDecodeError, jsonschema.ValidationError) as e:
            errors[model] = [str(e)]
```

 Every LLM that is expected to return structured JSON must have its output validated against the contract schema on every response, not just in tests. Schema failures are caught at the application layer, not the user layer.



AI Coach John

Follow



 Repost

test_refusal_consistency Policy Alignment Check

```

REFUSAL_MARKERS = [
    "I cannot", "I'm unable", "I don't", "I won't",
    "I can't assist", "not able to help"
]

async def test_refusal_consistency(
    boundary_queries: list[str],
    models: list[str],
    llm_client
) → list[dict]:
    mismatches = []
    for query in boundary_queries:
        refusals = {}
        for model in models:
            answer = await llm_client.complete(model, query)
            is_refusal = any(m in answer for m in REFUSAL_MARKERS)
            refusals[model] = is_refusal

        # All models should agree on refusal or all should answer
        unique_decisions = set(refusals.values())
        if len(unique_decisions) > 1:
            mismatches.append({

```

Why This Is Critical

If Claude refuses a borderline query and GPT-4o answers it, users quickly learn which model to use to bypass restrictions. Refusal inconsistency is a safety policy hole, not just a quality issue.



ConsistencyGuardedRouter Wrapper Pattern

```

class ConsistencyGuardedRouter:
    def __init__(self, primary: str, fallbacks: list[str], llm_client):
        self.primary = primary
        self.fallbacks = fallbacks
        self.client = llm_client

    async def complete(self, query: str, run_consistency=True) → dict:
        # Always get the primary model response
        primary_resp = await self.client.complete(self.primary, query)

        if not run_consistency:
            return {"response": primary_resp, "consistency_checked": False}

        answers = {self.primary: primary_resp}
        for model in self.fallbacks:
            answers[model] = await self.client.complete(model, query)

        sem_score, verdict = score_semantic_consistency(answers, embedder)
        fact_score, contra = check_factual_consistency(answers)

        if contra:
            log_for_review(query, answers, contra)

        return {
            "response": primary_resp,
            "consistency_verdict": verdict,
            "semantic_score": sem_score,
            "contradictions": contra,
        }

```



ModelRegressionSuite Baseline Snapshots

```
import json

class ModelRegressionSuite:
    def __init__(self, embedder):
        self.baselines: dict = {} # model → query → answer
        self.embedder = embedder

    def capture_baselines(self, models, golden_queries, llm_client):
        for model in models:
            self.baselines[model] = {}
            for q in golden_queries:
                resp = llm_client.complete_sync(model, q)
                self.baselines[model][q] = resp
            log(f"Captured baselines for {len(models)} models")

    def run_regression(self, models, golden_queries, llm_client) → dict:
        regressions = []
        for model in models:
            for q in golden_queries:
                new_resp = llm_client.complete_sync(model, q)
                base_emb = self.embedder.embed(self.baselines[model][q])
                new_emb = self.embedder.embed(new_resp)
                sim = np.dot(base_emb, new_emb)
                if sim < 0.85:
                    regressions.append({"model": model, "query": q, "sim": sim})
        return {"regressions": regressions, "total": len(regressions)}
```



Always capture baseline snapshots BEFORE upgrading any model version. Run the regression suite immediately after the upgrade. Any similarity below 0.85 is flagged for human review before traffic is switched.



Production Consistency Monitoring Schedule

Check	Frequency	Alert Threshold	Action
Semantic score	Hourly on 50 queries	Below 0.85	Route to primary model only
Factual score (NLI)	Per request on sensitive topics	Any contradiction	Reject and log for review
Schema validation	Every structured response	Any schema failure	Return 422 error, log model
Refusal consistency	Daily on 20 boundary queries	Any mismatch	Review routing policy
Regression suite	Every model deploy	Similarity below 0.85	Block deploy, review responses

 **Cost vs. Coverage Trade-off**

Running NLI on every response is expensive. Use semantic similarity as a fast first filter. Only run NLI on responses where semantic similarity is between 0.75 and 0.90, the region where contradictions are most likely to hide behind similar vocabulary.





COMPARISON

Consistency Approaches Comparison

Approach	What It Catches	Cost	Latency
Semantic similarity	Paraphrase drift, topic divergence, major stylistic differences between models	Low (embedding call)	Under 10ms
NLI contradiction check	Factual contradictions, logical inconsistencies hidden behind similar vocabulary	Medium (cross-encoder)	50–100ms
Schema validation	Output format violations, missing required fields, type errors in structured outputs	Negligible (CPU only)	Under 1ms
Human review	Everything, including subtle factual errors and tone inconsistencies	Very high (labor cost)	Hours to days



Recommended Stack

Schema validation on every response (free). Semantic similarity on all responses in batch (cheap). NLI only on the 10–15% of responses in the ambiguous similarity range. Human review on flagged contradictions only.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

Repost

Multi-Model Consistency Mistakes

- ✖ **Routing to different models without consistency checking.** Users routed to different models get different answers. Without checks, contradictions accumulate silently until a user notices and complains publicly.
- ✖ **Assuming the same prompt gives semantically equivalent answers across models.** GPT-4o and Claude have different training data, RLHF preferences, and output styles. Same prompt, same context, meaningfully different answers on many queries.
- ✖ **No output schema validation.** Each model formats JSON slightly differently even with the same system prompt. Without jsonschema, frontend failures are random and hard to debug.
- ✖ **Testing refusal on one model but not all.** Refusal policies differ across providers and model versions. A query refused by GPT-4o may be answered by Claude or an older GPT version.
- ✖ **No baseline snapshots before model upgrades.** Without pre-upgrade snapshots, you cannot quantify how much behavior changed after an upgrade. You are flying blind on regression risk.
- ✖ **Using semantic similarity as the only consistency metric.** Two answers can have 0.93 cosine similarity but state opposite facts using similar vocabulary. NLI is required to catch factual contradictions.



Multi-Model Consistency **Full Reference**

Component	Code	Threshold
Semantic consistency	<code>score_semantic_consistency()</code>	Above 0.85
Factual consistency	<code>check_factual_consistency()</code>	Zero contradictions
Schema validation	<code>validate_output_contract()</code>	Zero schema failures
Refusal consistency	<code>test_refusal_consistency()</code>	All-or-none refusal per query
Regression detection	<code>ModelRegressionSuite.run_regression()</code>	Similarity above 0.85 vs baseline
Guarded routing	<code>ConsistencyGuardedRouter.complete()</code>	Route to primary on INCONSISTENT
NLI model	<code>cross-encoder/nli-deberta-v3-small</code>	Under 100ms latency on CPU

== Every model in your pool must agree on facts, format, and boundaries.
Inconsistency is a product bug.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

 KEY TAKEAWAY

Consistent Models, Trustworthy Products

Users do not know which model answered their question. They just know if the product gives them reliable, consistent information. Consistency checking across your model pool is what makes multi-model routing a feature rather than a liability.

Start Today: 3 Actions

- Add `validate_output_contract()` to every endpoint that uses multiple models. Deploy this week, zero performance impact.
- Run `score_semantic_consistency()` daily on your top 50 queries across all routed models and alert on scores below 0.85.
- Capture baseline snapshots with `ModelRegressionSuite` before the next model version upgrade. Run regression check after the upgrade before switching traffic.

Follow AI Coach John for more production AI engineering content.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost